

MULTI-MODAL CNN BASED STROKE DETECTION FROM CT IMAGES AND CLINICAL DATA

A PROJECT REPORT

Submitted by

KANISKA M P

SANTHIYA K

SARUMATHI A

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

CHETTINAD COLLEGE OF ENGINEERING & TECHNOLOGY

ANNA UNIVERSITY:: CHENNAI 600025

MAY 2025

MULTI-MODAL CNN BASED STROKE DETECTION FROM CT IMAGES AND CLINICAL DATA

A PROJECT REPORT

Submitted by

KANISKA M P

SANTHIYA K

SARUMATHI A

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

CHETTINAD COLLEGE OF ENGINEERING & TECHNOLOGY

ANNA UNIVERSITY :: CHENNAI 600025

MAY 2025

ANNA UNIVERSITY:: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report “**MULTI-MODAL CNN BASED STROKE DETECTION FROM CT IMAGES AND CLINICAL DATA**” is the bonafide work of “**KANISKA M P (920221104020), SANTHIYA K (92021104043), SARUMATHI A (920221104044)**”, who carried out the project work under my supervision.

SIGNATURE

Dr. R. Punithavathi B.E., M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor & Head

Department of Computer Science

and Engineering,

Chettinad College of Engineering

& Technology, Puliur CF,

Karur-639114.

SIGNATURE

Dr. R. Punithavathi B.E., M.E., Ph.D.,

SUPERVISOR

Professor & Head

Department of Computer Science

and Engineering,

Chettinad College of Engineering

& Technology, Puliur CF,

Karur-639114.

Submitted for University Project Viva – Voce held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

At the outset, we express our thanks to our parents for providing us the best atmosphere for doing this project in our career.

We extend our deep gratitude to our Vice Chairman **Shri.M.A.M.R.Muthiah** and President **Tmt.Geetha Muthiah** for providing us the good infrastructure facilities and resources.

We thank our Secretary **Ms.Trayee Sinha**, for permitting us to proceed with the project within the campus.

We wish to express our gratitude to our Principal, **Dr.(Mrs).A.Punitha**, for her continuous encouragement.

We wish to convey our special thanks to **Mr. R. Rajkumar**, Program co-ordinator, Department of Computer Science and Engineering, for his support and providing us ample time to complete our project.

We express our indebtedness to the supervisor of our project, **Dr.(Mrs) R. Punithavathi B.E., M.E., Ph.D** Professor & Head, Department of Computer Science and Engineering for her guidance throughout the course of our project.

Our sincere thanks to **all the teaching and non-teaching staff members** of Chettinad College of Engineering & Technology, Karur and beloved thanks to our friends for their help in the successful completion of this project work.

Finally we bow before God, the Almighty who always has a better plan for us. We give our praise and glory to the Almighty in all our endeavours.

KANISKAM P

SANTHIYA K

SARUMATHI A

ABSTRACT

Existing stroke outcome models often focus only on neuroimaging or clinical data separately, limiting prediction accuracy. This project integrates both brain CT images and clinical variables for a more comprehensive outcome prediction. U-Net is used to segment important brain regions from CT scans to focus on relevant anatomical structures. 3D DenseNet¹⁶⁹ extracts deep spatial features from the segmented CT images for stroke prognosis. A Random Forest algorithm is applied to clinical data such as age, BMI, and blood pressure to predict outcomes. A multi-stage model is introduced: Model A (image-based prediction), Model B (clinical data-based prediction), and Model C (combined decision-making for ambiguous cases). The fusion of imaging and clinical features improves prediction accuracy, outperforming traditional single-modality models.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	IV
	TABLE OF CONTENTS	V
	LIST OF FIGURES	VIII
	LIST OF ABBREVIATIONS	IX
1	INTRODUCTION	1
	1.1 OBJECTIVE	1
	1.2 SCOPE	2
2	FEASIBILITY STUDY	4
	2.1 EXISTING SYSTEM	4
	2.1.1 Disadvantages	5
	2.2 PROPOSED SYSTEM	6
	2.2.1 Advantages	7
	2.3 SYSTEM SPECIFICATION	8
	2.3.1 Hardware Requirement	8
	2.3.2 Software Requirement	8
	2.3.3 OpenCV	9
	2.3.4 Scikit-learn	9
	2.3.5 Matplotlib	9

3	LITERATURE SURVEY	11
4	SYSTEM DESIGN	16
	4.1 BLOCK DIAGRAM	16
5	SYSTEM IMPLEMENTATION	18
	5.1 LIST OF MODULES	18
	5.2 MODULES DESCRIPTION	18
	5.2.1 Project Kick-Off	18
	5.2.2 Research and Technology Assessment	19
	5.2.3 Prototype Development and Testing	19
	5.2.4 Pilot Implementation in Selected Cities	20
6	SYSTEM TESTING	21
	6.1 UNIT TESTING	22
	6.2 FUNCTIONAL TESTING	22
	6.3 INTEGRATION TESTING	23
7	UML DIAGRAMS	24
	7.1 INTRODUCTION	24
	7.2 GOALS	25
	7.3 USE CASE DIAGRAM	26
	7.4 CLASS DIAGRAM	28
	7.5 SEQUENCE DIAGRAM	29

	7.6 ACTIVITY DIAGRAM	31
	7.7 COLLABORATION DIAGRAM	32
8	CONCLUSION & FUTURE WORK	34
	8.1 CONCLUSION	34
	8.2 FUTURE WORK	35
9	APPENDICES	36
	9.1 SOURCE CODE	36
	9.2 SCREENSHOTS	44
	REFERENCES	47

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Stroke Outcome Prediction System Using CT Images and Clinical Data	26
7.1	Use Case Diagram	36
7.2	Class Diagram	38
7.3	Sequence Diagram	40
7.4	Activity Diagram	41
7.5	Collaboration Diagram	43

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the Curve
BMI	Body Mass Index
CNN	Convolutional Neural Network
CT	Computed Tomography
CSV	Comma-Separated Values
ECG	Electrocardiogram
GPU	Graphics Processing Unit
LIME	Local Interpretable Model-Agnostic Explanations
LSTM	Long Short-Term Memory
mRS	Modified Rankin Scale
OpenCV	Open-Source Computer Vision Library
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SVM	Support Vector Machine
U-NET	U-shaped Convolutional Neural Network (Segmentation model)

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The objective of this project is to develop an intelligent system that predicts stroke patient outcomes by integrating brain CT images and clinical data. The system automates the Modified Rankin Scale (mRS) scoring process, classifying patients into favourable (mRS 0–2) and unfavourable (mRS 3–6) outcome categories. It uses a U-Net model for brain image segmentation, a 3D DenseNet169 architecture for image feature extraction, and a Random Forest algorithm for analysing clinical data such as age, BMI, and blood pressure. By combining predictions from both image-based and clinical models, the system aims to provide a more accurate and reliable prognosis compared to traditional manual assessments.

The project introduces a multi-stage prediction system with Model A (image-based), Model B (clinical data-based), and Model C (combination model for uncertain cases) to further enhance prediction accuracy. The goal is to minimize human dependency, reduce diagnostic variability, and enable faster, standardized outcome predictions. This system is designed to be scalable, adaptable to various hospital settings, and easily integrable into clinical workflows, ultimately improving the efficiency of stroke management and supporting healthcare professionals in critical decision-making.

Additionally, the system aims to assist rural and under-resourced hospitals where experienced neurologists may not be readily available. By providing a quick and automated prognosis, the tool can help guide immediate treatment strategies and prioritize patients for specialized care when needed.

Furthermore, this project lays the foundation for future research in AI-driven healthcare applications. It encourages the development of more sophisticated models that can handle diverse medical imaging modalities and

clinical scenarios, contributing to the advancement of precision medicine and improving overall patient outcomes.

1.2 SCOPE

The scope of this project covers the design, development, and evaluation of a deep learning-based system for stroke outcome prediction using both imaging and clinical data. The project focuses on integrating brain CT scan analysis and patient clinical parameters to predict the Modified Rankin Scale (mRS) outcome efficiently and accurately.

The system specifically targets binary classification — favourable versus unfavourable outcomes — to simplify decision-making in real-world hospital environments. It is built using a U-Net model for image segmentation, 3D DenseNet169 for feature extraction from CT images, and a Random Forest classifier for clinical data analysis. By combining these outputs through a multi-stage model approach, the system enhances prediction accuracy and minimizes errors associated with using a single data source.

This project is scalable and can be extended to support a wider variety of datasets and clinical parameters. While initially trained on a limited set of features like age, BMI, and blood pressure, the system architecture allows easy retraining to include additional patient data such as glucose levels, smoking history, or cholesterol levels. Future versions can also incorporate MRI images or other diagnostic modalities to broaden its applicability across different neurological conditions.

In addition, the system is intended for integration into hospital management software, telemedicine platforms, and rural healthcare programs. By making stroke prognosis tools accessible beyond major hospitals, the project supports equitable healthcare delivery and promotes the use of artificial intelligence in medical practice. It also opens opportunities for academic research and continuous innovation in the field of AI-based clinical outcome prediction.

CHAPTER 2

FEASIBILITY STUDY

A feasibility study is conducted to assess the practicality, viability, and benefits of implementing the proposed stroke outcome prediction system. It ensures that the project can be successfully developed within the available technological, operational, and financial resources. The study covers various aspects including technical feasibility, operational feasibility, and economic feasibility to ensure that the system can be effectively deployed in real-world healthcare environments.

Technical Feasibility: The project is technically feasible due to the availability of advanced machine learning frameworks, powerful neural network models like U-Net and Dense Net, and established libraries for medical image processing. Cloud computing platforms and high-performance hardware such as GPUs make training deep learning models efficient and manageable. Open-source medical datasets and simulation environments further support the development and validation of the system.

Operational Feasibility: From an operational standpoint, the system is designed to be user-friendly and integrate seamlessly into hospital workflows. Healthcare providers can easily input CT images and clinical data through a simple interface and receive quick predictions, minimizing disruptions to existing processes. By assisting doctors in prognosis and treatment planning, the system enhances operational efficiency, reduces manual workload, and improves patient care outcomes.

Economic Feasibility: The project is economically viable as it reduces the long-term costs associated with manual diagnosis errors, delayed treatments, and prolonged hospital stays. Initial investments in model development and hardware can be justified by the significant improvements in diagnostic speed and accuracy. Moreover, by making the system adaptable and scalable, it can be reused and updated for various medical conditions without needing complete

redevelopment, offering strong long-term value. This feasibility study confirms that the proposed system is not only practical but also beneficial for implementation in clinical environments, supporting better stroke management and enhancing the quality of healthcare delivery.

2.1 EXISTING SYSTEM

Currently, stroke outcome prediction largely relies on traditional methods involving clinical examination, manual scoring, and radiologist interpretation of CT or MRI scans. Physicians typically use clinical scoring systems like the National Institutes of Health Stroke Scale (NIHSS) or manually assign a Modified Rankin Scale (mRS) score based on observed patient symptoms and imaging results. The process is heavily dependent on the experience and judgment of medical professionals, which can lead to variability and subjective bias.

Radiologists manually review CT brain images to detect early signs of ischemia, haemorrhage, or brain swelling. However, subtle changes in CT images can be difficult to detect, particularly in the early stages of stroke, leading to delayed or inaccurate diagnosis. Furthermore, clinical outcome predictions often require the integration of multiple factors such as age, comorbidities, blood pressure, and other vitals, which is usually done manually or with basic statistical tools, limiting the speed and consistency of the assessment.

Some healthcare institutions have introduced decision support systems based on simple algorithms, but these systems are limited in their predictive power and lack the ability to process complex imaging and clinical data together. As a result, outcome predictions may not be sufficiently accurate or timely to guide critical treatment decisions, particularly in fast-paced environments like emergency rooms or stroke centres.

Additionally, existing systems often lack the ability to learn and improve over time based on new data. Manual assessments remain static and are not updated automatically as new patient outcomes are recorded. This prevents continuous model refinement, making it difficult for hospitals to leverage growing databases of medical records for better future predictions. Overall, there is a strong need for the solution that can automate, standardize, and continuously improve stroke prognosis.

2.1.1 Disadvantages

- They depend heavily on manual interpretation of CT scans, which can vary from one medical expert to another, leading to inconsistent results.
- There is no automatic process to analyze or compare the scans with past patient data, making diagnosis slower and less accurate.
- Important clinical factors such as blood pressure, age, or glucose levels are often not considered alongside the imaging data, resulting in incomplete analysis.
- The process requires more time and effort from healthcare professionals, especially in emergency cases where quick decisions are critical.
- Many existing tools are outdated, paper-based, or rely on limited software that doesn't support modern diagnostic improvements.
- There is no integration between clinical records and CT scan data, so the decision-making process lacks a full view of the patient's condition.
- The current systems do not offer predictive support for doctors to anticipate outcomes or recovery levels.
- They are not user-friendly and are often not accessible in rural hospitals or primary health centers due to lack of proper infrastructure.

2.2 PROPOSED SYSTEM

The proposed system aims to overcome the limitations of the existing stroke detection and prediction methods by introducing an automated, multi-stage solution that integrates both brain CT scan analysis and clinical data processing. Instead of depending on manual interpretation, the system uses structured techniques to extract relevant patterns and information from patient CT images and vital medical parameters. This helps in identifying stroke regions more accurately and predicting patient outcomes with better precision.

The first part of the system focuses on segmenting the brain CT images to isolate the affected areas. This segmentation helps remove background noise and ensures only the relevant brain regions are analysed. Once the brain region is extracted, it is processed using advanced feature extraction methods that identify patterns such as tissue damage, blood clot visibility, or structural deformities. These features help in assessing the severity and location of the stroke.

Parallely, clinical information such as age, gender, blood pressure, glucose level, cholesterol level, body mass index (BMI), and previous medical history is collected and processed. These variables are essential in understanding the overall health condition of the patient and how likely they are to recover from the stroke. The proposed system uses structured decision-making algorithms to evaluate these inputs and generate outcome predictions, such as whether a patient is likely to have a favourable or unfavourable recovery.

A unique part of the system is the multi-stage model architecture. The system includes:

- **Model A:** Focuses on image-based outcome prediction using segmented CT scan data.
- **Model B:** Analyzes clinical information to provide an independent outcome prediction.
- **Model C:** Acts as a final decision-maker that combines the outputs of Model A and B, especially when the predictions are unclear or conflicting.

This multi-model approach ensures that predictions are not only based on a single type of data but from multiple reliable sources, enhancing the trust and robustness of the final decision. Furthermore, this system can assist healthcare professionals by offering a second opinion in emergency scenarios where time is crucial. Since the system is designed with a user-friendly structure, it can also be used in regional hospitals and healthcare centers with minimal training.

In summary, the proposed system offers a well-integrated, faster, and more accurate alternative to traditional methods. It reduces dependency on manual interpretation, leverages both image and clinical insights, and enhances the overall diagnosis and stroke outcome prediction process. This leads to better planning for treatment, quicker intervention, and more informed decision-making for healthcare providers.

2.2.1 Advantages

- **Improved Accuracy:** By combining CT scan features and clinical data, the system ensures more accurate outcome predictions.
- **Faster Diagnosis Support:** Automated analysis speeds up the decision-making process in critical emergency situations.
- **Structured Decision-Making:** A multi-model approach (image, clinical, combined) provides balanced, well-rounded predictions.
- **Considers Multiple Patient Factors:** Vital parameters like blood pressure, age, and glucose levels are included for deeper analysis.
- **Reduced Human Error:** Automated processes minimize mistakes from manual interpretation.
- **User-Friendly Interface:** Simple design allows even small hospitals and clinics to use it effectively.
- **Better Planning for Treatment:** Doctors can plan timely and appropriate treatment and recovery strategies based on predictions.

2.3 SYSTEM SPECIFICATION

2.3.1 Hardware Requirements

- Processor: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Storage: At least 500 GB HDD or 256 GB SSD
- Display: 15.6” HD Monitor or better for viewing brain scans clearly.
- Graphics Card (optional): A basic GPU (e.g., NVIDIA GTX 1050)

2.3.2 Software Requirements

- Operating System: Windows 10/11, Linux (Ubuntu), or macOS — based on user preference and software compatibility.
- Programming Environment: Python 3.8 or higher (for data processing and image analysis)
- Google Colab – for cloud-based development, testing, and running models without local installation
- **Libraries & Tools:**
 - OpenCV – for CT image processing
 - NumPy & Pandas – for data manipulation
 - Matplotlib or Seaborn – for visualizations
 - Scikit-learn – for clinical data-based prediction
- **Storage:** SQLite or CSV-based lightweight storage for clinical records and prediction outputs.
- **Others:**
 - Google Drive or local folder access for importing/exporting patient CT data
 - Git & GitHub – for version control and collaboration.

2.3.3 OpenCV (OpenSource Computer Vision Library)

OpenCV is crucial for preprocessing CT brain images, which is a foundational step before feeding them into the CNN model. This involves tasks such as:

- **Grayscale Conversion:** CT images are typically in grayscale; ensuring consistent format is important.
- **Noise Removal:** Filtering techniques (like Gaussian or median filtering) reduce image noise and improve segmentation accuracy.
- **Resizing & Normalization:** Standardizing the input size (e.g., 256x256) and pixel values (0-1 range) ensures uniformity for the CNN model.
- **Contour Detection or Region Isolation:** Helps focus the model on the brain region by removing background or irrelevant structures.

2.3.4 Scikit-learn (Machine Learning Library)

- **Purpose in the Project:** Scikit-learn is used to build and evaluate predictive models based on clinical data (age, blood pressure, glucose, etc.). In your system:
- **Random Forest or Logistic Regression:** Can be used to classify stroke outcomes based on numerical and categorical inputs.
- **Data Splitting & Cross-Validation:** Ensures robust training/testing and prevents overfitting.
- **Performance Metrics (e.g., Accuracy, ROC-AUC, Confusion Matrix):** Provide insights into how well the clinical model performs.

2.3.5 Matplotlib (Visualization Library)

Matplotlib is an essential library used in this project for visualizing data, monitoring model performance, and interpreting results. It supports detailed and customizable plotting, helping to gain deeper insights into both the clinical data and the model's behavior. Key applications include:

- **Data Distribution Analysis:** Used to create histograms and bar plots that illustrate distributions of clinical variables such as age, blood pressure, and glucose level.
- **Model Performance Monitoring:** Plots of accuracy and loss curves during training help detect overfitting or underfitting in CNN and machine learning models.
- **Confusion Matrix Visualization:** Helps in interpreting classification performance by visually comparing predicted vs actual outcomes.
- **ROC and AUC Curve Plotting:** Used to assess the performance of classification models by showing the trade-off between true positive rate and false positive rate.

CHAPTER 3

LITERATURE SURVEY

The study titled "Evaluating Performance of EEG Data-Driven Machine Learning for Traumatic Brain Injury Classification" by Nicolas Vivaldi (2021) explores the application of electroencephalogram (EEG) data in conjunction with machine learning techniques to classify traumatic brain injury (TBI). Utilizing the Temple EEG Corpus, the researchers implemented models such as Support Vector Machines (SVM) and Convolutional Neural Networks (CNN) to detect patterns indicative of TBI. The models achieved high classification accuracy, reaching up to 94%, highlighting the effectiveness of EEG signals in diagnosing brain injuries. This work underscores the potential of EEG as a non-invasive, accessible diagnostic tool, especially valuable in clinical settings where advanced imaging technologies may not be readily available. Furthermore, the study demonstrates the broader utility of machine learning in enhancing diagnostic capabilities for various neurological disorders.

Disadvantages:

Limited Sample Size and Diversity: The study relied on the Temple EEG Corpus, which may not encompass a sufficiently broad demographic or clinical variety. This limits the generalizability of the model to real-world, heterogeneous populations.

The study titled "AI-Based Stroke Disease Prediction System Using ECG and PPG Bio-Signals" by JaeHak Yu (2022) presents a machine learning-driven approach for predicting stroke risk using electrocardiogram (ECG) and photoplethysmogram (PPG) signals. By utilizing wearable sensor technology, the system enables continuous, non-invasive, and real-time monitoring of patients' bio-signals. The proposed model integrates Convolutional Neural Network (CNN) layers for spatial feature extraction and Long Short-Term Memory (LSTM) networks to capture temporal patterns in the signals. This hybrid architecture effectively identifies subtle physiological changes that may indicate an increased risk of stroke. Achieving a high accuracy of 99.15% on test data, the system demonstrates strong potential for early stroke detection. Moreover, its support for remote health monitoring makes it particularly beneficial for elderly individuals and populations in rural areas with limited access to advanced diagnostic tools. The study highlights the promise of combining artificial intelligence with telemedicine to improve preventive healthcare and broaden access to timely medical interventions.

Disadvantages:

- Noisy ECG and PP signals may lead to misinterpretation and false predictions. The model was trained on a limited dataset, potentially reducing its generalization capability across diverse populations and real-world conditions.

The study titled "Automated Stroke Prediction Using Machine Learning: An Explainable and Exploratory Study with a Web Application for Early Intervention" by Krishna Mridha (2023) introduces a stroke prediction system that utilizes machine learning for early diagnosis and real-time clinical decision-making. A key feature of this system is its integration of explainable AI techniques, specifically SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations), which enhance the transparency and interpretability of the model's predictions. By analyzing essential clinical features such as age, hypertension, heart disease, glucose levels, and lifestyle habits, the model achieves a prediction accuracy of 91%. The system is made accessible through a user-friendly web application, allowing clinicians to visualize individual patient risk factors and understand the underlying reasons for each prediction. This fosters greater trust in the AI-driven approach and supports more informed medical decisions, particularly in emergency and telemedicine environments.

Disadvantages:

- Data bias and lack of diversity in training datasets can affect prediction reliability. Limited generalization of the model to unseen or complex clinical scenarios.

The study titled "Machine Learning and Acute Stroke Imaging" by Sunil A. Sheth (2023) reviews the application of machine learning techniques within acute stroke imaging workflows. It emphasizes how AI models can significantly enhance stroke detection, automate image interpretation, and facilitate rapid clinical decision-making. The review assesses the performance of various models in critical tasks such as identifying ischemic regions, quantifying lesion volumes, and predicting patient outcomes based on CT and MRI scans. By reducing diagnostic delays and minimizing variability in interpretation among radiologists, machine learning has the potential to improve the efficiency and accuracy of stroke management. Furthermore, the study underscores the clinical importance of integrating imaging data with AI algorithms to enable timely and precise interventions in stroke care.

Disadvantages:

- The effectiveness of AI models is highly dependent on the quality and consistency of training datasets. Interpretability of complex models remains a barrier to clinical trust and adoption.

The study titled "An Improved Concatenation of Deep Learning Models for Predicting and Interpreting Ischemic Stroke" by Sunil A. Sheth (2023) delves into the integration of machine learning techniques within acute stroke imaging workflows. It highlights the pivotal role of artificial intelligence in enhancing the detection of stroke, automating the interpretation of medical images, and enabling rapid clinical decision-making. The review evaluates a range of AI models for key tasks such as identifying ischemic regions, quantifying lesion volumes, and predicting patient outcomes using CT and MRI scan data. By leveraging these advanced techniques, the study demonstrates AI's ability to reduce diagnostic delays, decrease variability in assessments among radiologists, and improve the overall quality of stroke management. Furthermore, it emphasizes the clinical value of combining imaging data with AI algorithms to ensure timely and accurate interventions in stroke care.

Disadvantages:

- The effectiveness of AI models is highly dependent on the quality and consistency of training datasets. Interpretability of complex models remains a barrier to clinical trust and adoption.

CHAPTER 4

SYSTEM DESIGN

4.1 BLOCK DIAGRAM

The proposed system predicts stroke outcomes by combining CT scan images and patient clinical data through a multi-stage process. It begins with brain CT slices processed by a U-Net segmentation module to isolate stroke-affected regions. The segmented images are then passed into a 3D DenseNet-169 model (Model A) to extract features and predict the initial outcome. Simultaneously, clinical data such as age, blood pressure, and BMI are analyzed by a Random Forest model (Model B). The results from both models are compared and passed to a fusion module (Model C), which finalizes the prediction. If the models agree, the outcome is accepted; otherwise, Model C re-evaluates using both image and clinical inputs. The final output classifies the stroke as favourable (mRS 0–2) or unfavourable (mRS 3–6), ensuring a reliable and balanced prediction. The complete workflow of the proposed system is illustrated in Figure 4.1.

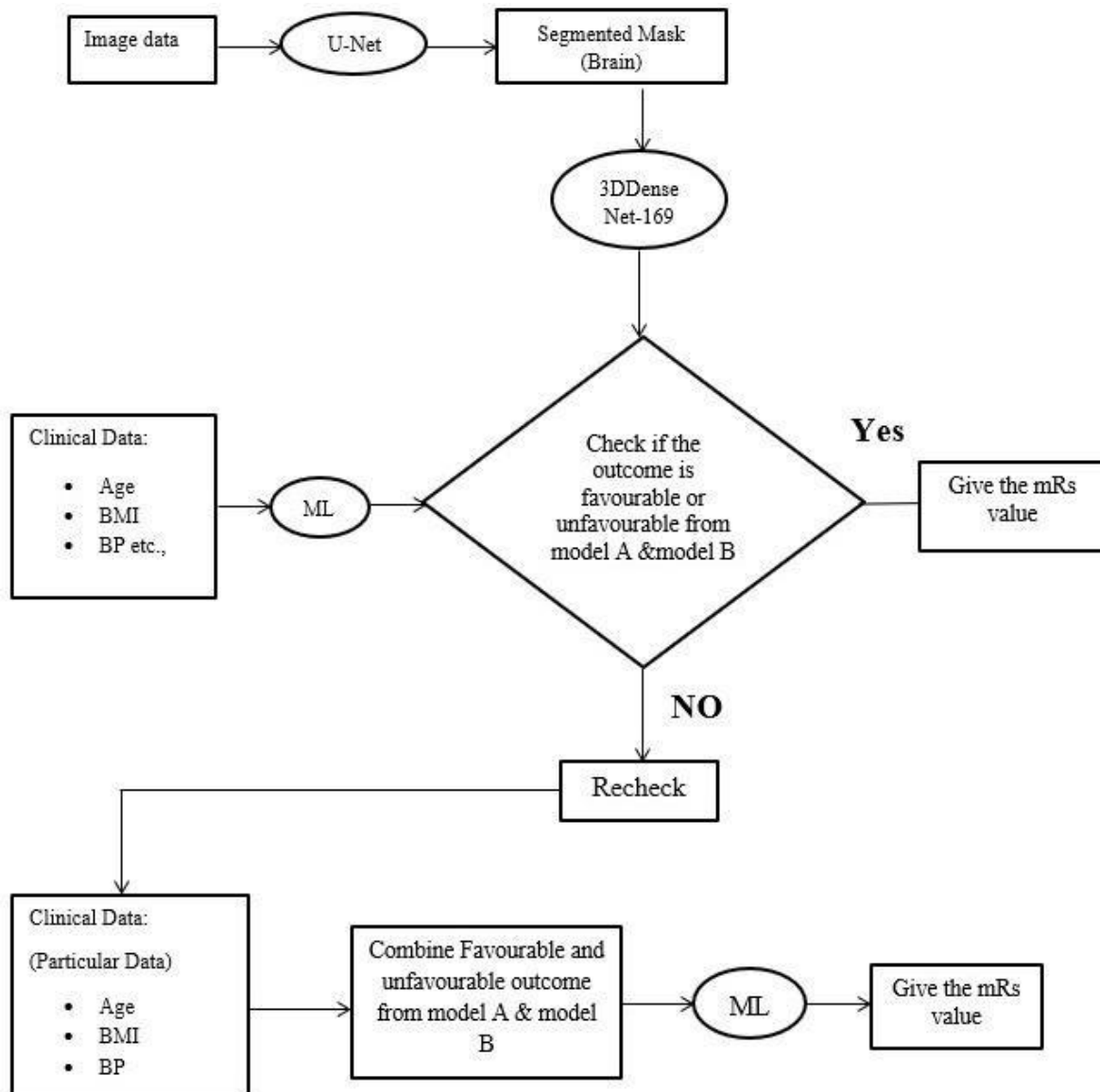


Fig.4.1 Block Diagram of Stroke Outcome Prediction System Using CT Images and Clinical Data

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 LIST OF MODULES

Module 1: Project Kick-Off

Module 2: Research and Technology Assessment

Module 3: Prototype Development and Testing

Module 4: Pilot Implementation in Selected Cities

5.2 MODULES DESCRIPTION

5.2.1 Module 1: Project Kick-Off

This module serves as the foundation of the entire project, focusing on planning and team coordination. The primary goal of the project was defined as predicting stroke outcomes using brain CT images and patient clinical data. Roles and responsibilities were assigned to each team member to ensure efficient task distribution and accountability. Suitable data sources were identified, including publicly available medical imaging datasets and clinical records. The necessary tools and technologies such as Python, Google Colab, and OpenCV were selected to support development and testing. A version control system was established using GitHub to manage code updates and team collaboration. Additionally, a basic user interface layout was designed to facilitate input of both CT images and patient clinical information for further processing.

5.2.2 Module 2: Research and Technology Assessment

In this module, a thorough analysis of existing research and relevant technologies was conducted to identify the best-suited methods for stroke outcome prediction. Various deep learning and machine learning models were evaluated to determine their effectiveness in processing medical images and clinical data. After comparing segmentation models, U-Net was chosen for its proven performance in medical image segmentation. For outcome classification based on 3D CT images, 3D DenseNet-169 was selected due to its ability to handle volumetric data and extract deep spatial features. The team also explored different algorithms for clinical data processing, eventually finalizing Random Forest for its accuracy and interpretability with structured health data. Platforms such as Google Colab were assessed for model training, offering an efficient cloud-based environment. This module ensured the technology choices aligned with the project goals and were feasible within the team's timeline and resources.

5.2.3 Module 3: Prototype Development and Testing

This module involved the core development phase where each component of the system was implemented and tested. The team began by training the U-Net model on CT scan images to accurately segment brain regions affected by stroke. Next, the 3D DenseNet-169 model was developed to analyze the segmented image volumes and classify outcomes as favourable or unfavourable. A separate Random Forest model was trained on clinical data such as age, BMI, and blood pressure to provide a parallel prediction. Integration of these models was done through a decision fusion mechanism (Model C), which combined the results for the final prediction. The prototype was then tested on a small dataset to evaluate performance in terms of accuracy, consistency, and speed. Any errors or inconsistencies were debugged, and adjustments were made to improve

model behaviour. Visual tools like confusion matrices and ROC curves were used to analyze results.

5.2.4 Module 4: Pilot Implementation in Selected Cities

In the final module, the prototype system was tested in a simulated or controlled environment that mimicked real-world clinical conditions. Selected test cases, resembling actual patient data, were used to validate the full pipeline from input to prediction. CT scan volumes and corresponding clinical data were uploaded, and predictions were monitored for consistency. The goal of this stage was to observe the system's behavior under practical scenarios and ensure that it worked reliably with varied input. Performance metrics such as prediction time, output clarity, and usability were recorded. Based on the feedback and results, minor improvements were made to the user interface and prediction logic. This pilot testing helped confirm the readiness of the system for potential deployment in hospitals, diagnostic centers, or research environments.

CHAPTER 6

SYSTEM TESTING

Testing is a critical phase in the software development lifecycle that ensures the functionality, reliability, and performance of the system. For the Multi-Modal CNN-Based Stroke Detection System, various testing techniques are employed to verify the accuracy of the models, validate the integration of CT image processing and clinical data analysis modules, and assess the overall robustness of the system. The primary objective of the testing phase is to identify and correct defects, ensure seamless module interaction, and confirm that the system meets the specified requirements.

Testing Objectives:

The main objectives of the testing phase in this project include:

- Verifying that each module (image processing, clinical data analysis, and decision fusion) functions correctly as per the design specifications.
- Ensuring data accuracy and consistency in the CT scan analysis and clinical data processing modules.
- Validating the integration of models (U-Net, DenseNet169, and Random Forest) to confirm that they work seamlessly together.
- Assessing the system's performance in terms of prediction accuracy, processing speed, and memory usage.
- Identifying and resolving defects to minimize false positives/negatives in stroke outcome predictions.

The experimental analysis of the complexity of an algorithm or an experimental comparison of two or more algorithms, which solve the same problem.

There are three ways to test a program

- For Correctness
- For Implementation efficiency
- For Computational Complexity

Tests for correctness are supposed to verify that a program does exactly what it was designed to do. This is much more difficult than it may at first appear, especially for large programs. The data is entered in all forms separately and whenever an error occurs, it is corrected immediately. A quality team deputed by the management verified all the necessary documents and tested the Software while entering the data at all levels. The development process involves various types of testing. Each test type addresses a specific testing requirement. The most common types of testing involved in the development process are:

- Unit Test
- Functional Test
- Integration Test

6.1 Unit Testing

The first test in the development process is the unit test. The source code is normally divided into modules, which in turn are divided into smaller units called units. These units have specific behavior. The test done on these units of code is called unit test. Unit test depends upon the language on which the project is developed. Unit tests ensure that each unique path of the project performs accurately to the documented specifications and contains clearly defined inputs and expected results.

6.2 Functional Testing

Functional test can be defined as testing two or more modules together with the intent of finding defects, demonstrating that defects are not present, verifying that the module performs its intended functions as stated in the specification and establishing confidence that a program does what it is supposed to do. It is a type of software testing which is used to verify the functionality of the software application, whether the function is working according to the requirement specification. In functional testing, each function tested by giving the value, determining the output, and verifying the actual

output with the expected value. Functional testing performed as black-box testing which is presented to confirm that the functionality of an application or system behaves as we are expecting. It is done to verify the functionality of the application.

6.3 Integration Testing

In integration testing modules are combined and tested as a group. Modules are typically code modules, individual applications, source and destination applications on a network, etc. Integration Testing follows unit testing and precedes system testing. Testing after the product is code complete. Betas are often widely distributed or even distributed to the public at large in hopes that they will buy the final product when it is released. The focus of the integration testing level is to expose defects at the time of interaction between integrated components or units. Integration Testing is a division of software testing that tests interfaces between different software components. Any software module will work well individually, but when it's integrated with a different module, there are chances where the software might not behave as intended.

CHAPTER 7

UML DIAGRAMS

7.1 INTRODUCTION

Unified Modelling Language (UML) is a standardized visual language used to model the structure and behaviour of a system through graphical representations. It provides a comprehensive framework for visualizing, specifying, constructing, and documenting the components of software systems. By depicting system architecture, UML diagrams enable developers and stakeholders to understand complex system structures and interactions effectively.

UML diagrams are categorized into structural diagrams and behavioural diagrams. Structural diagrams, such as class and object diagrams, represent the static aspects of the system, including data objects, attributes, and their relationships. Behavioural diagrams, including sequence, activity, and use case diagrams, illustrate the dynamic flow of control and data among system components.

In the context of the Multi-Modal CNN-Based Stroke Detection System, UML diagrams play a crucial role in modelling the interaction between different modules, such as CT scan preprocessing, clinical data analysis, and prediction report generation. They provide a visual blueprint of how patient data flows through the system, how image data is segmented and processed, and how predictions are generated and displayed.

7.2 GOALS

The application of UML in the Multi-Modal CNN-Based Stroke Detection System is integral to effectively designing and documenting the complex architecture of the system. The key goals include:

Defining System Architecture:

- UML diagrams provide a structured representation of the system's architecture, illustrating the interaction between various modules such as CT Scan processing, Clinical Data analysis, Prediction Model, and Prediction Report.

- This ensures that each component's responsibilities and interactions are clearly defined, preventing ambiguity and promoting modular design.

- Visualizing Data Flow and Interactions:

- The use of sequence diagrams and collaboration diagrams helps in visualizing the dynamic flow of data during prediction generation.

- This is particularly crucial in a multi-modal system where clinical data and image data are processed simultaneously, requiring precise synchronization and data exchange.

- Enhancing Communication Among Stakeholders:

- UML serves as a universal language that bridges the gap between technical and non-technical stakeholders, such as developers, data scientists, and radiologists.

- By representing system components visually, UML diagrams facilitate effective communication, ensuring that all parties have a shared understanding of the system structure and functionality.

- Identifying Dependencies and Relationships:

- UML diagrams, especially class diagrams, effectively capture the relationships and dependencies between different classes and data entities.

- This is particularly relevant in the stroke detection system, where patient data, CT scan data, and clinical records are interlinked and must be accurately

mapped to avoid data inconsistencies.

- **Supporting System Analysis and Testing:**

By clearly outlining the interactions between objects and modules, UML diagrams aid in identifying potential testing scenarios and data flows.

- For instance, the activity diagram can outline the flow of operations from data input to report generation, helping in pinpointing critical points for testing and validation.

- **Ensuring Scalability and Maintainability:**

- A well-structured UML model promotes a modular design, allowing for future expansions without affecting the core architecture.

- New components, such as additional data processing modules or advanced predictive algorithms, can be easily integrated into the system using the established UML structure.

7.3 USE CASE DIAGRAM

A Use Case Diagram provides a high-level overview of the system by illustrating the interactions between various actors (users) and system functionalities. It helps in defining the key use cases that represent the primary operations in the stroke detection system. The corresponding Use Case Diagram is shown in Figure 7.1.

Purpose in the Stroke Detection System:

- Identifies the main users (actors) such as Radiologist, Data Analyst, and Hospital Administrator.
- Maps out core functionalities such as uploading CT scans, processing clinical data, generating predictions, and viewing reports.
- Establishes the flow of interactions between the actors and system modules.

Key Use Cases:

- Upload CT Scan Images
- Process Clinical Data
- Perform Image Segmentation
- Generate Prediction
- View Prediction Report
- Manage Patient Records

Actors:

- Radiologist: Uploads scans and views prediction reports.
- Data Analyst: Processes clinical data and generates predictions.
- Hospital Administrator: Manages patient records and monitors system reports.

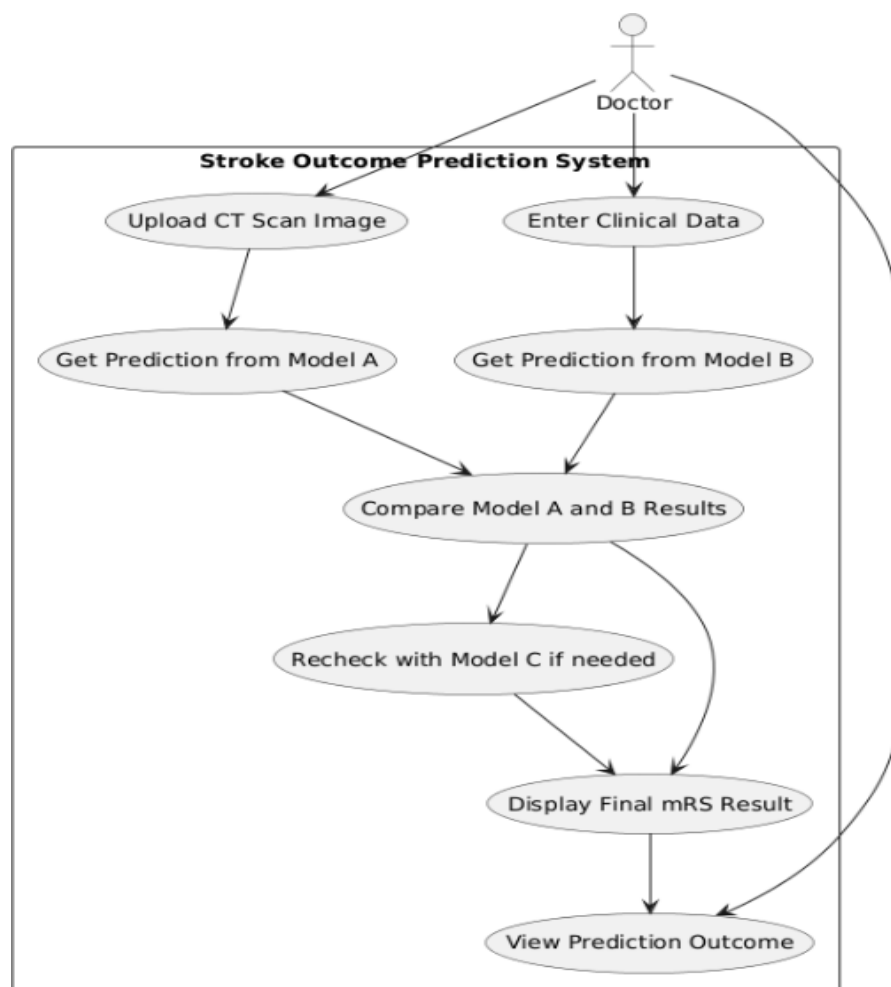


Fig.7.1 Use Case Diagram

7.4 CLASS DIAGRAM

Class Diagram defines the static structure of the system by illustrating the classes, their attributes, methods, and relationships. It provides a comprehensive view of the system's data model and how various components interact. The corresponding Class Diagram is shown in Figure 7.2.

Purpose in the Stroke Detection System:

- Represents key classes such as Patient, CTScan, ClinicalData, PredictionModel, PredictionReport, and DatabaseManager.
- Specifies attributes (e.g., patientID, scanID) and methods (e.g., uploadScan(), generateReport()).
- Establishes relationships such as associations, dependencies, and generalizations.

Key Classes and Their Roles:

- Patient: Stores patient information and links to scan data and clinical records.
- CTScan: Manages image data and performs segmentation.
- ClinicalData: Processes and analyzes patient health records.
- PredictionModel: Integrates image and clinical data to generate predictions.
- PredictionReport: Stores and manages generated reports.
- DatabaseManager: Handles data storage and retrieval for all classes.

Relationships:

- Patient → CTScan (1-to-many)
- Patient → ClinicalData (1-to-many)
- CTScan, ClinicalData → PredictionModel (data inputs)
- PredictionModel → PredictionReport (output generation)

- All classes → DatabaseManager (data storage and retrieval)

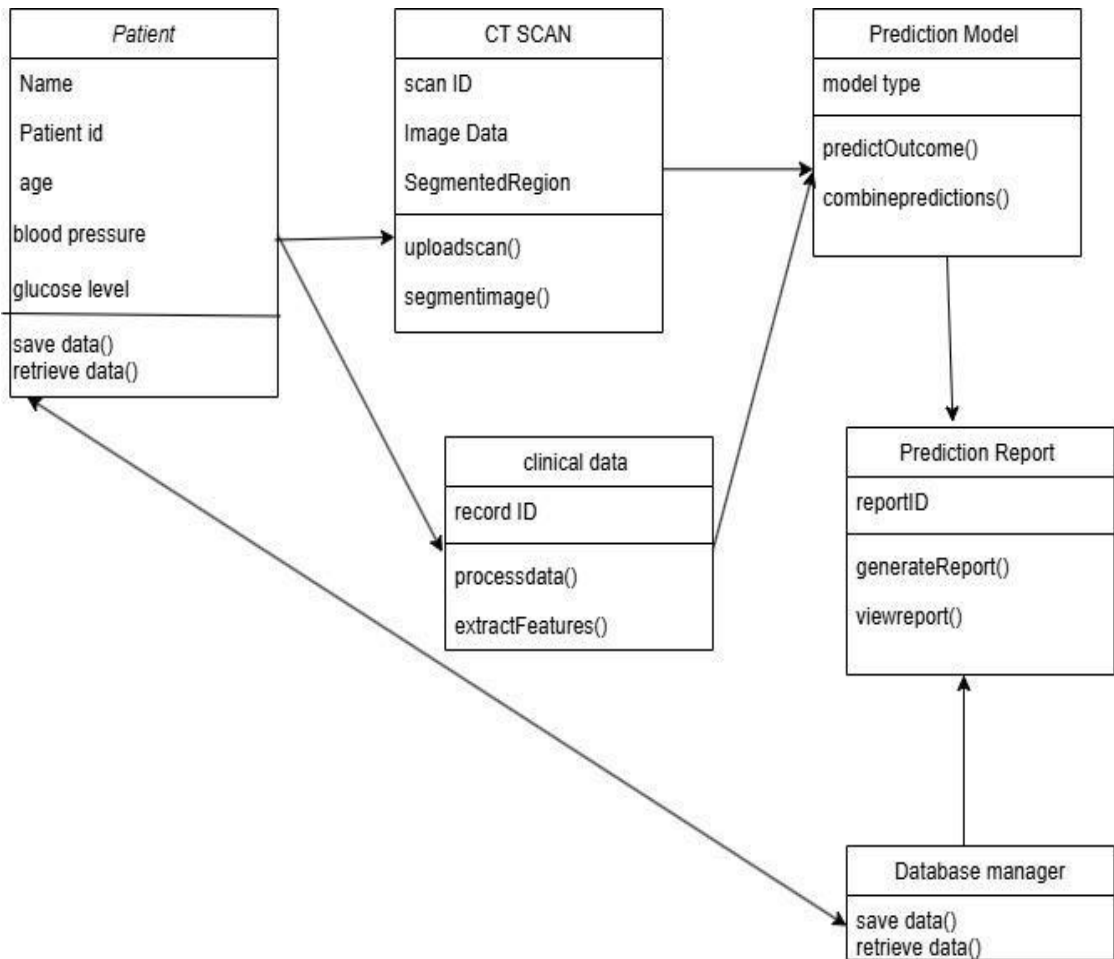


Fig.7.2 Class Diagram

7.5 SEQUENCE DIAGRAM

A Sequence Diagram illustrates the flow of data and the sequence of interactions between objects over time. It is particularly useful in understanding the dynamic flow of operations in the stroke detection process. The corresponding Sequence Diagram is shown in Figure 7.3.

Purpose in the Stroke Detection System:

- Demonstrates the flow of interactions when a stroke prediction report is generated.
- Visualizes the order of method calls, such as uploadScan(), processData(), and generateReport().
- Highlights how data flows between components like CTScan, ClinicalData, and PredictionModel.

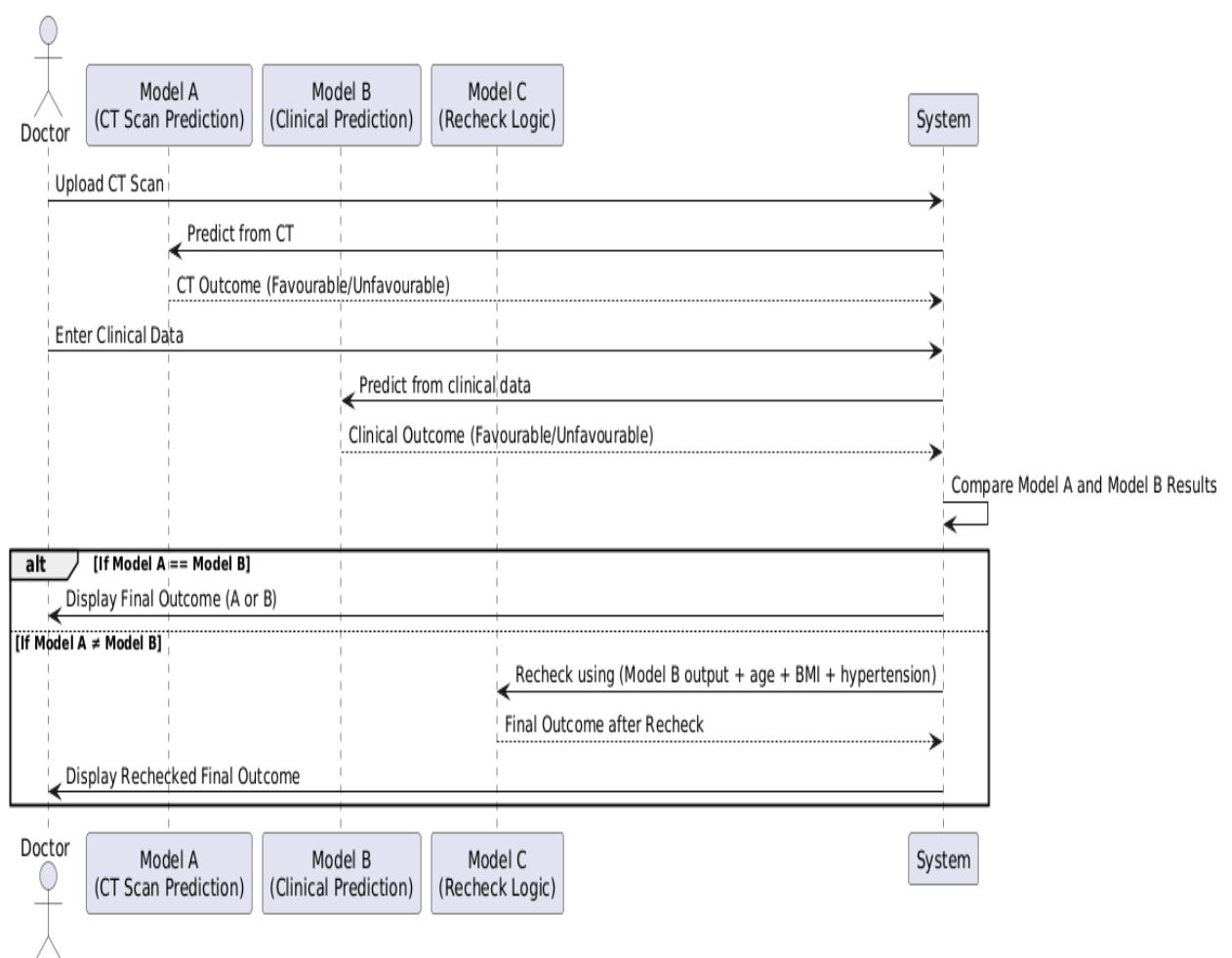


Fig.7.3 Sequence Diagram

Sequence of Interactions:

- Radiologist uploads a CT scan (uploadScan() to CTScan).
- CTScan processes the scan and extracts features (segmentImage()).
- Data Analyst inputs clinical data (processData() to ClinicalData).
- ClinicalData sends processed data to PredictionModel (extractFeatures()).
- PredictionModel integrates both data inputs (predictOutcome() to PredictionReport).
- PredictionReport generates the report and stores it (generateReport() and saveData() to DatabaseManager).
- Radiologist retrieves the report (viewReport() from PredictionReport).

7.6 ACTIVITY DIAGRAM

An Activity Diagram represents the flow of activities and decisions within a system, emphasizing the sequence of actions and decision points. It is particularly useful in visualizing workflows and control logic. The corresponding Activity Diagram is shown in Figure 7.4.

Purpose in the Stroke Detection System:

- Outlines the step-by-step process of generating a stroke prediction report, from data input to report generation.
- Highlights concurrent activities, such as simultaneous processing of CT scan images and clinical data.
- Identifies decision points, such as data validation and report generation.

Flow of Activities:

1. Start: System initializes and waits for data input.
2. Input Patient Data: Radiologist inputs patient details.
3. Upload CT Scan: Radiologist uploads a CT scan.
4. Preprocess Image: CTScan module segments the brain region.
5. Process Clinical Data: Data Analyst inputs clinical data.

6. Generate Prediction: PredictionModel integrates both data input and generates the prediction.
7. Store Prediction Report: PredictionReport stores the generated report.
8. Display Report: Radiologist views the prediction report.
9. End: Process completed

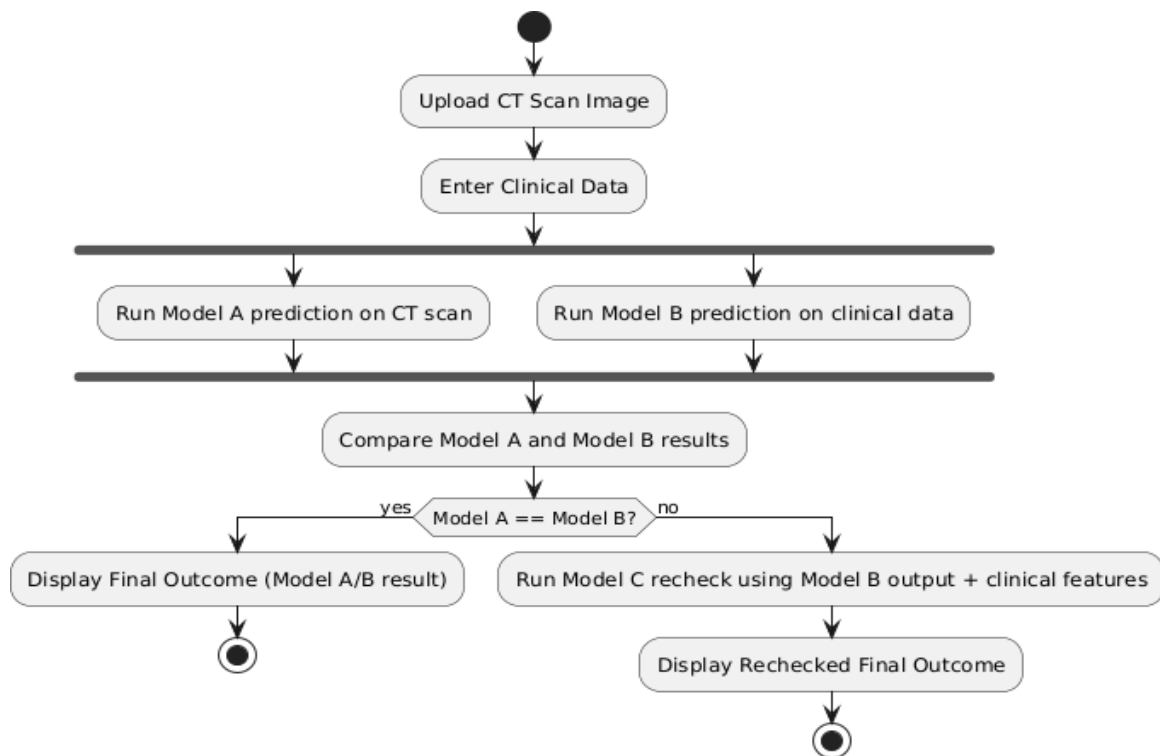


Fig.7.4 Activity Diagram

7.7 COLLABORATION DIAGRAM

A Collaboration Diagram (or Communication Diagram) focuses on the structural organization of objects and how they interact through message exchanges. It highlights the relationships between objects and the sequence of messages exchanged. The corresponding Collaboration Diagram is shown in Figure 7.5.

Purpose in the Stroke Detection System:

- Illustrates the interaction between objects such as Radiologist, CTScan, ClinicalData, and PredictionModel during the prediction generation process.
- Identifies message exchanges and the flow of data across system modules.
- Emphasizes the structure and sequence of communications, aiding in understanding the data exchange flow.

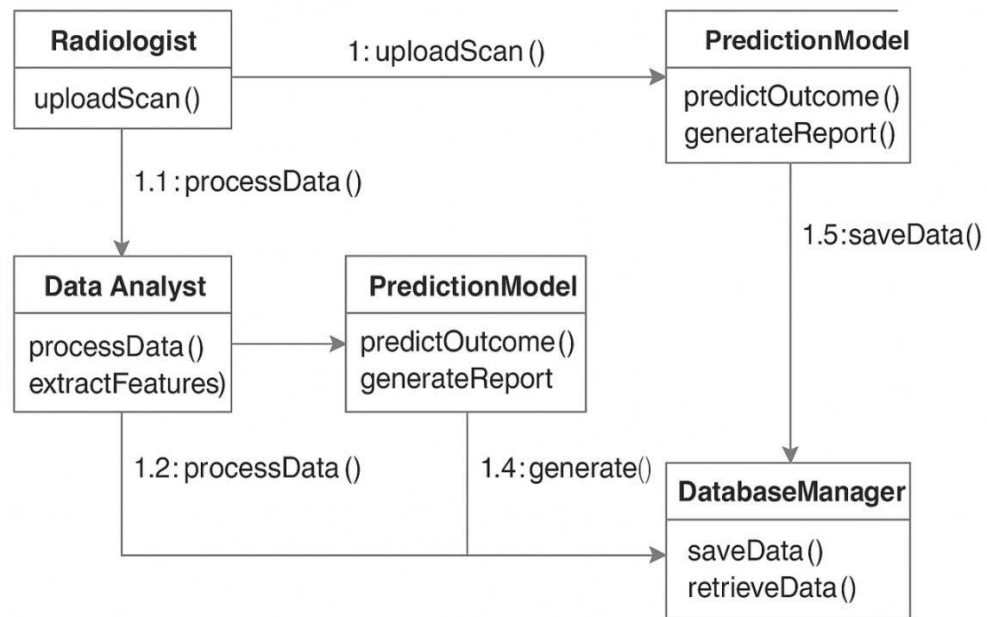


Fig.7.5 Collaboration Diagram

Objects and Interactions:

- Radiologist → CTScan: uploadScan()
- CTScan → PredictionModel: processScan()
- Data Analyst → ClinicalData: processData()
- ClinicalData → PredictionModel: extractFeatures()
- PredictionModel → PredictionReport: generateReport()
- PredictionReport → DatabaseManager: saveData()

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

The proposed Multi-Modal CNN-Based Stroke Detection System successfully integrates both medical imaging and clinical data analysis to enhance the accuracy of stroke outcome prediction. By leveraging convolutional neural networks (CNNs) for CT scan analysis and machine learning models for clinical data processing, the system demonstrates the potential to provide a comprehensive diagnostic framework for early stroke detection.

The image processing module effectively segments stroke-affected regions using techniques such as U-Net and DenseNet169, enabling precise identification of ischemic and haemorrhagic strokes. Simultaneously, the clinical data analysis module processes vital parameters like age, blood pressure, and glucose levels using Random Forest and Logistic Regression, resulting in additional predictive insights.

The integration of both data modalities through a decision fusion model enhances diagnostic accuracy, reducing false positives and false negatives. Moreover, the use of SQLite for data management ensures efficient storage and retrieval of patient records and prediction reports. The implementation of OpenCV, Scikit-learn, and Matplotlib further streamlines data preprocessing, model training, and result visualization.

Overall, the proposed system demonstrates significant potential for improving stroke detection outcomes, particularly in clinical settings where timely diagnosis is crucial. However, further refinements are necessary to enhance model robustness and generalization capabilities.

8.1 FUTURE WORK

Future work for the Multi-Modal CNN-Based Stroke Detection System includes expanding the dataset to improve model accuracy and reduce overfitting. Incorporating advanced data augmentation techniques and integrating MRI scans and blood test reports can further enhance diagnostic capabilities.

Optimizing the model for real-time deployment using TensorFlow Lite and developing a user-friendly interface will facilitate clinical adoption. Implementing explainability techniques like Grad-CAM can aid clinicians in interpreting predictions, while clinical trials can validate the model's effectiveness using real-world data.

CHAPTER 9

APPENDICES

9.1 SOURCE CODE

(i) Model A:

```
from google.colab import files

from PIL import Image

import torch

import torchvision.transforms as transforms

import torch.nn as nn

from torchvision import models


# Step 1: Upload a new image

uploaded = files.upload()

image_path = list(uploaded.keys())[0]


# Step 2: Load the trained binary model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = models.densenet169(weights=None)

num_fts = model.classifier.in_features
```

```

    model.classifier = nn.Sequential(

nn.Linear(num_fts, 1),

nn.Sigmoid()

)

```

```

model.load_state_dict(torch.load("/content/drive/MyDrive/final_project/model
A_densenet169.pth", map_location=device))

model = model.to(device)

model.eval()

```

Step 3: Define transform

```

transform = transforms.Compose([

transforms.Resize((224, 224)),

transforms.ToTensor(),

transforms.Normalize([0.485, 0.456, 0.406],[0.229, 0.224, 0.225]))

```

Step 4: Predict outcome and approximate mRS

```

def predict_outcome_with_mrs(image_path, model, transform,
threshold=0.5):

img = Image.open(image_path).convert('RGB')

img_tensor = transform(img).unsqueeze(0).to(device)

```

```

with torch.no_grad():

prob = model(img_tensor).item()

    if prob < 0.5:

        if prob < 0.2:

            mrs = 0

        elif prob < 0.35:

            mrs = 1

        else:

            mrs = 2

return f" Favourable Outcome (mRS 0–2) → Predicted mRS: {mrs}
[Confidence: {1 - prob:.2f}]"

else:

    if prob < 0.65:

        mrs = 3

    elif prob < 0.8:

        mrs = 4

        elif prob < 0.9:

            mrs = 5

        else:

            mrs = 6

return f" Unfavourable Outcome (mRS 3–6) → Predicted mRS: {mrs}

```

```
[Confidence: {prob:.2f}]"
```

```
# Step 5: Run prediction
```

```
result = predict_outcome_with_mrs(image_path, model, transform)
```

```
print("Prediction Result:", result)
```

(ii) Model B & C:

```
import pandas as pd
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
# === Load Clinical Data for Model B and Fusion Recheck ===
```

```
clinical_data = pd.read_csv('/content/drive/MyDrive/final_project/
```

```
clinical_data_with_mrs.csv')
```

```
clinical_data['gender'] = clinical_data['gender'].map(
```

```
{'Male': 1, 'Female': 0})
```

```
clinical_data['smoking_status'] = clinical_data['smoking_status'].map
```

```
({'Smoker': 1, 'Non-Smoker': 0})
```

```
# === Model B Features and Labels ===
```

```
features_B = ['age', 'bmi', 'hypertension', 'heart_disease', 'avg_glucose_level',  
'gender', 'smoking_status']
```

```
X_B = clinical_data[features_B]
```

```
y_B = clinical_data['mrs']
```

```

# === Train Model B ===

modelB = RandomForestClassifier

(n_estimators=100, random_state=42)modelB.fit(X_B, y_B)


# === Get User Clinical Input ===

print(" Enter Clinical Data for Prediction")

age = float(input("Age: "))

bmi = float(input("BMI: "))

hypertension = int(input("Hypertension? (1: Yes, 0: No): "))

= int(input("Heart Disease? (1: Yes, 0: No): "))

avg_glucose_level = float(input("Average Glucose Level: "))

gender = int(input("Gender? (1: Male, 0: Female): "))

smoking_status =

int(input("Smoking Status? (1: Smoker, 0: Non-    Smoker): "))


input_B = pd.DataFrame([{

'age': age,

'bmi': bmi,

'hypertension': hypertension,

'heart_disease': heart_disease,

'avg_glucose_level': avg_glucose_level,

```

```

        'gender': gender,

        'smoking_status': smoking_status

    })

```

```

# === Predict with Model B ===

```

```

modelB_mrs = modelB.predict(input_B)[0]

```

```

print(f"\n Model B Predicted mRS:

```

```

{modelB_mrs} → {'Favourable'

```

```

if modelB_mrs <= 2 else 'Unfavourable'}")

```

```

# === Enter Model A Output Manually ===

```

```

modelA_mrs =

```

```

int(input("\n Enter mRS from Model A (0 to 6): "))

```

```

print(f"Model A Output →

```

```

mRS {modelA_mrs} →

```

```

{'Favourable' if modelA_mrs <= 2 else    'Unfavourable'}")

```

```

# === Recheck if Conflict Detected ===

```

```

if modelA_mrs > 2 and modelB_mrs <= 2:

```

```

print("\n Conflict Detected:

```

```

Model A = Unfavourable, Model B =    Favourable")

```

```
print("Rechecking with Fusion Model ( Age + BMI + Hypertension)...")
```

```
# Prepare Fusion Model
```

```
clinical_data['ModelB_mRS'] = modelB.predict(X_B)
```

```
fusion_features = ['ModelB_mRS', 'age', 'bmi', 'hypertension']
```

```
X_fusion = clinical_data[fusion_features]
```

```
y_fusion = clinical_data['mrs']
```

```
fusion_model =
```

```
RandomForestClassifier(n_estimators=100, random_state=42)
```

```
fusion_model.fit(X_fusion, y_fusion)
```

```
# Prepare Fusion Input
```

```
input_fusion = pd.DataFrame([{
```

```
    'ModelB_mRS': modelB_mrs,
```

```
    'age': age,
```

```
    'bmi': bmi,
```

```
    'hypertension': hypertension}])
```

```
final_mrs = fusion_model.predict(input_fusion)[0]
```

```
print(f"\n Final Decision after Recheck ➡
```

```

mRS = {final_mrs} →

{'Favourable' if final_mrs <= 2 else 'Unfavourable'})

else:

# Use Model A or B directly if no conflict

final_mrs =

modelA_mrs if modelA_mrs ==

modelB_mrs else max(modelA_mrs,    modelB_mrs)

print(f"\n Final Decision → mRS =

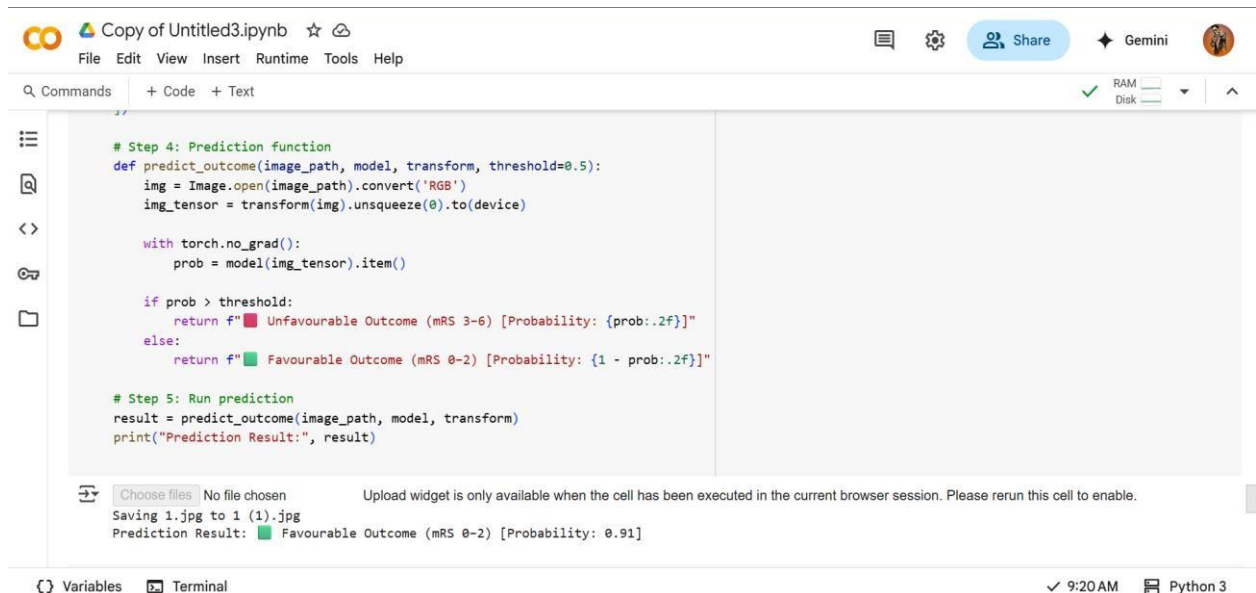
{final_mrs} → {'Favourable' if final_mrs <= 2 else 'Unfavourable'})

```

9.2 OUTPUT SCREENSHOTS

Case 1: (A=B)

If the Model A is Favourable.



The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code in the cell defines a function `predict_outcome` that takes an image path, a model, a transform, and a threshold. It opens the image, converts it to RGB, and uses the model to predict. If the probability is greater than the threshold (0.5), it returns "Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]". Otherwise, it returns "Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]". Below the function, the code runs the prediction for a specific image path. The output shows the prediction result: "Favourable Outcome (mRS 0-2) [Probability: 0.91]".

```
# Step 4: Prediction function
def predict_outcome(image_path, model, transform, threshold=0.5):
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0).to(device)

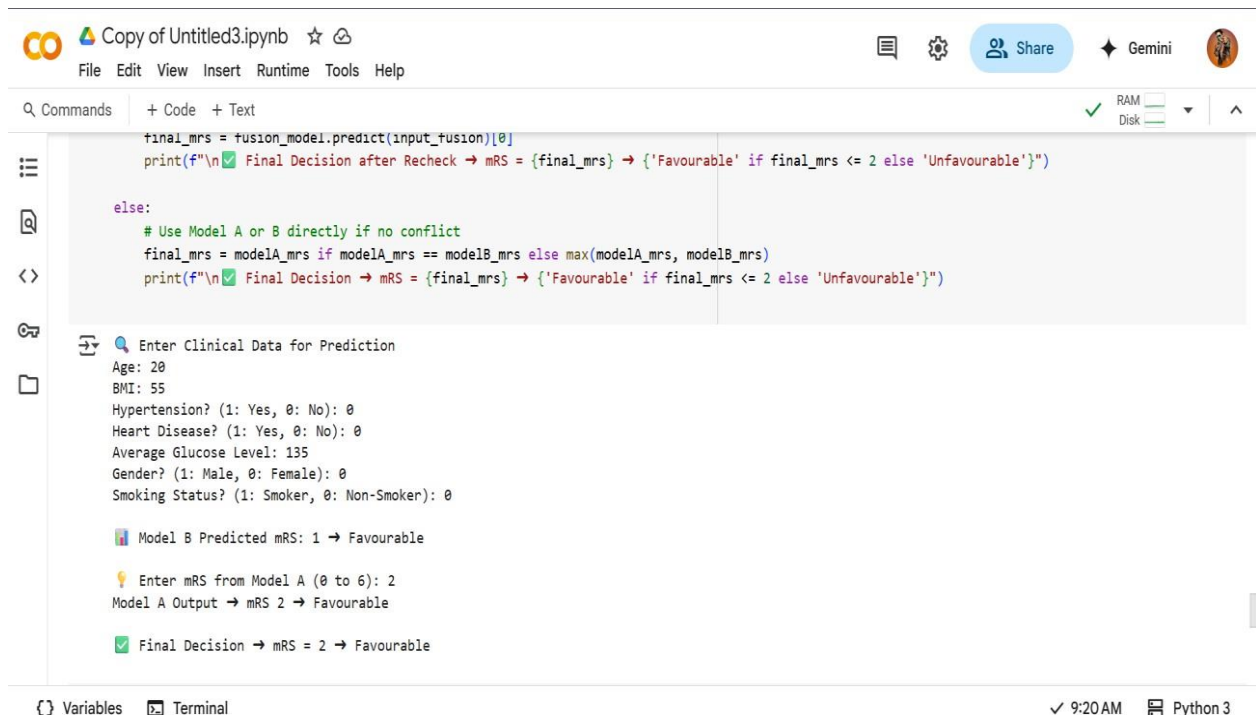
    with torch.no_grad():
        prob = model(img_tensor).item()

    if prob > threshold:
        return f"❌ Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]"
    else:
        return f"✅ Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]"

# Step 5: Run prediction
result = predict_outcome(image_path, model, transform)
print("Prediction Result:", result)
```

Prediction Result: ✅ Favourable Outcome (mRS 0-2) [Probability: 0.91]

If the Model B is Favourable.



The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code in the cell defines a function `predict_outcome` that takes an image path, a model, a transform, and a threshold. It opens the image, converts it to RGB, and uses the model to predict. If the probability is greater than the threshold (0.5), it returns "Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]". Otherwise, it returns "Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]". Below the function, the code runs the prediction for a specific image path. The output shows the prediction result: "Favourable Outcome (mRS 0-2) [Probability: 0.91]".

```
# Step 4: Prediction function
def predict_outcome(image_path, model, transform, threshold=0.5):
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        prob = model(img_tensor).item()

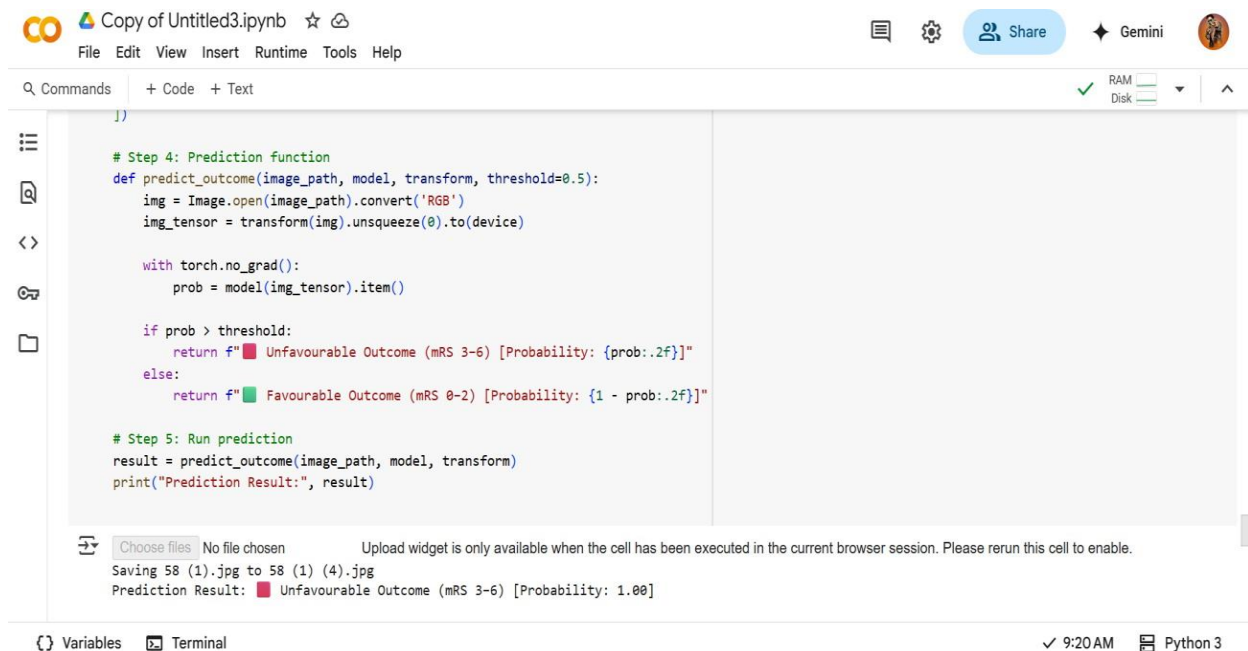
    if prob > threshold:
        return f"❌ Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]"
    else:
        return f"✅ Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]"

# Step 5: Run prediction
result = predict_outcome(image_path, model, transform)
print("Prediction Result:", result)
```

Prediction Result: ✅ Favourable Outcome (mRS 0-2) [Probability: 0.91]

Case 2: (A=B)

If the Model B is Unfavourable.

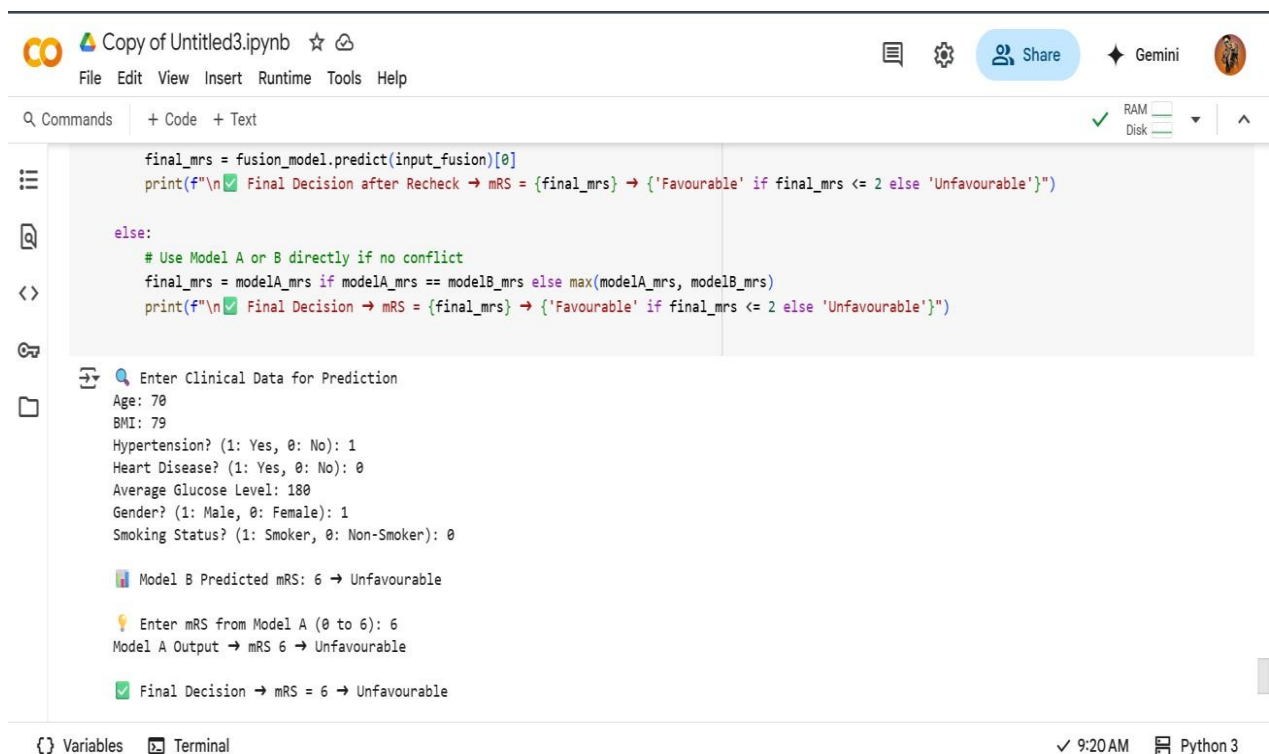


The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code defines a function `predict_outcome` that takes an image path, a model, and a transform as input. It converts the image to a tensor and uses the model to predict a probability. If the probability is greater than a threshold of 0.5, it returns an "Unfavourable Outcome (mRS 3-6)" with a probability of `prob:.2f`. Otherwise, it returns a "Favourable Outcome (mRS 0-2)" with a probability of `1 - prob:.2f`. The notebook also includes a "Step 5: Run prediction" section where the function is called with `image_path`, `model`, and `transform`. The output shows the prediction result: "Unfavourable Outcome (mRS 3-6) [Probability: 1.00]".

```
def predict_outcome(image_path, model, transform, threshold=0.5):  
    img = Image.open(image_path).convert('RGB')  
    img_tensor = transform(img).unsqueeze(0).to(device)  
  
    with torch.no_grad():  
        prob = model(img_tensor).item()  
  
    if prob > threshold:  
        return f"Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]"  
    else:  
        return f"Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]"  
  
# Step 5: Run prediction  
result = predict_outcome(image_path, model, transform)  
print("Prediction Result:", result)
```

Prediction Result: Unfavourable Outcome (mRS 3-6) [Probability: 1.00]

If the Model B is Unfavourable.



The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code defines a function `predict_outcome` that takes an image path, a model, and a transform as input. It converts the image to a tensor and uses the model to predict a probability. If the probability is greater than a threshold of 0.5, it returns an "Unfavourable Outcome (mRS 3-6)" with a probability of `prob:.2f`. Otherwise, it returns a "Favourable Outcome (mRS 0-2)" with a probability of `1 - prob:.2f`. The notebook also includes a "Step 5: Run prediction" section where the function is called with `image_path`, `model`, and `transform`. The output shows the prediction result: "Unfavourable Outcome (mRS 3-6) [Probability: 1.00]".

```
final_mrs = fusion_model.predict(input_fusion)[0]  
print(f"Final Decision after Recheck → mRS = {final_mrs} → {'Favourable' if final_mrs <= 2 else 'Unfavourable'}")  
  
else:  
    # Use Model A or B directly if no conflict  
    final_mrs = modelA_mrs if modelA_mrs == modelB_mrs else max(modelA_mrs, modelB_mrs)  
    print(f"Final Decision → mRS = {final_mrs} → {'Favourable' if final_mrs <= 2 else 'Unfavourable'}")
```

Enter Clinical Data for Prediction
Age: 70
BMI: 79
Hypertension? (1: Yes, 0: No): 1
Heart Disease? (1: Yes, 0: No): 0
Average Glucose Level: 180
Gender? (1: Male, 0: Female): 1
Smoking Status? (1: Smoker, 0: Non-Smoker): 0

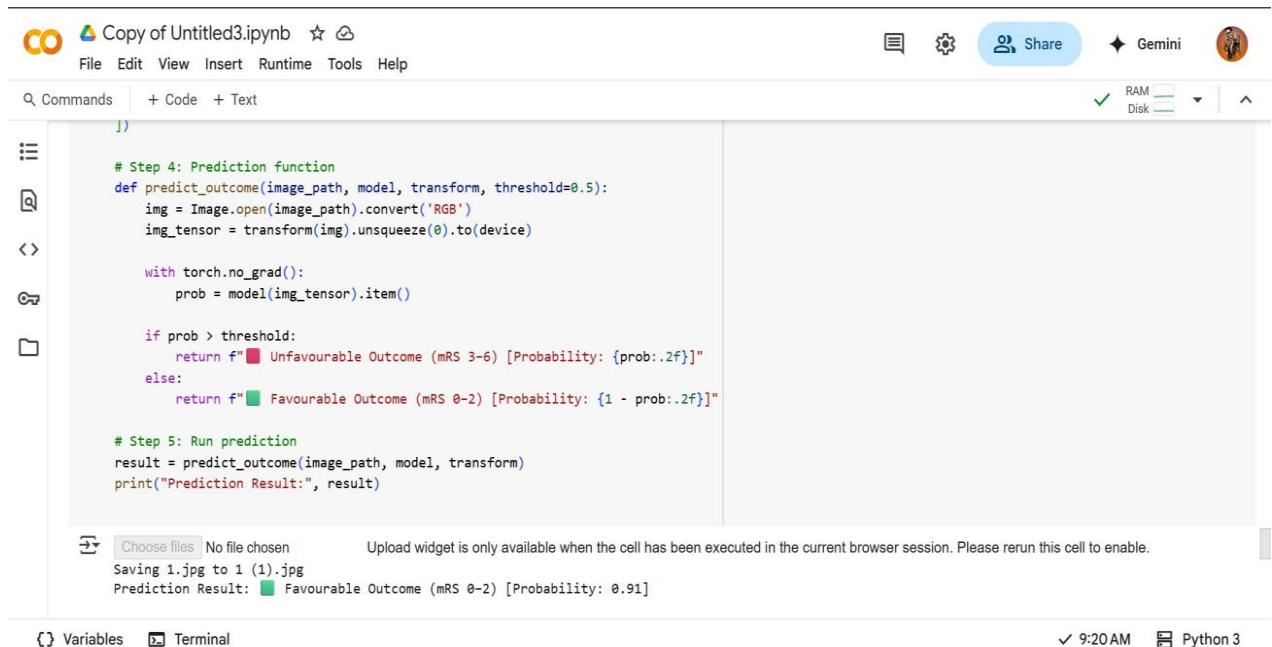
Model B Predicted mRS: 6 → Unfavourable

Enter mRS from Model A (0 to 6): 6
Model A Output → mRS 6 → Unfavourable

Final Decision → mRS = 6 → Unfavourable

Case 3: (A is not equal to B)

If the Model A is Favourable.



The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code defines a function `predict_outcome` that takes an image path, a model, a transform, and a threshold. It opens the image, converts it to RGB, and processes it with the model. If the probability is greater than the threshold (0.5), it returns an "Unfavourable Outcome (mRS 3-6)" with the probability. Otherwise, it returns a "Favourable Outcome (mRS 0-2)".

```
# Step 4: Prediction function
def predict_outcome(image_path, model, transform, threshold=0.5):
    img = Image.open(image_path).convert('RGB')
    img_tensor = transform(img).unsqueeze(0).to(device)

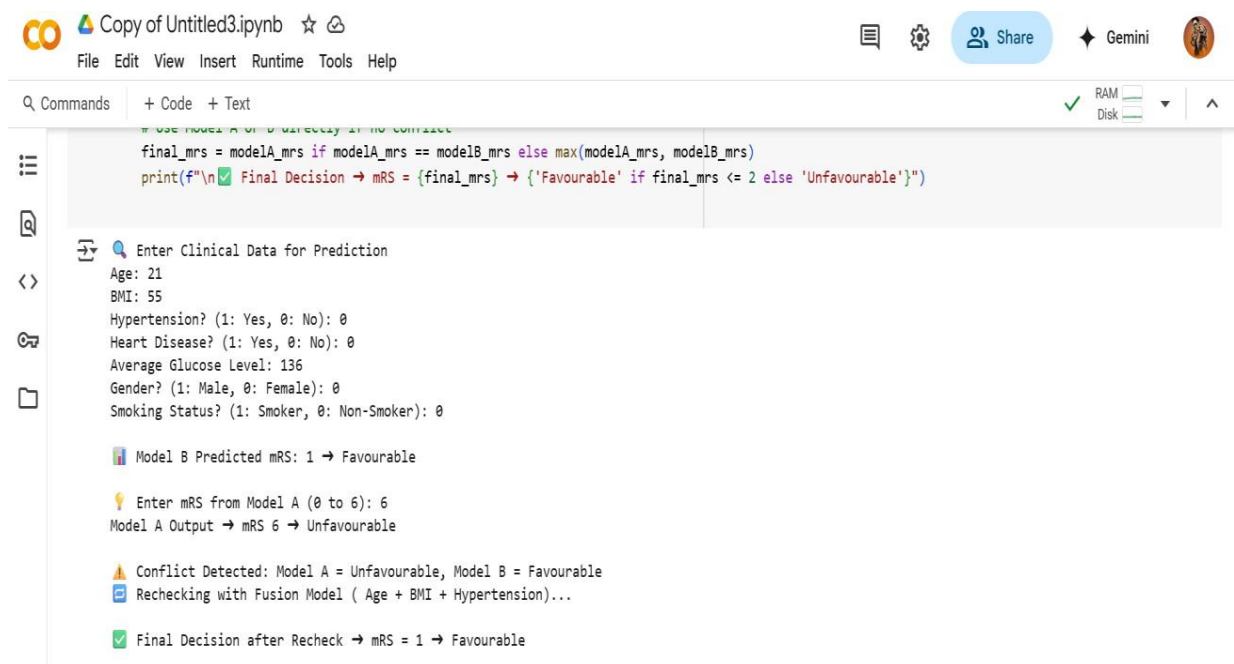
    with torch.no_grad():
        prob = model(img_tensor).item()

    if prob > threshold:
        return f"❌ Unfavourable Outcome (mRS 3-6) [Probability: {prob:.2f}]"
    else:
        return f"✅ Favourable Outcome (mRS 0-2) [Probability: {1 - prob:.2f}]"

# Step 5: Run prediction
result = predict_outcome(image_path, model, transform)
print("Prediction Result:", result)
```

The output of the cell shows: "Prediction Result: ✅ Favourable Outcome (mRS 0-2) [Probability: 0.91]".

If the Model B is Unfavourable.



The screenshot shows a Jupyter Notebook titled "Copy of Untitled3.ipynb". The code defines a function `final_mrs` that takes the mRS values from Model A and Model B. If they are the same, it returns that value. If they are different, it returns the maximum value. The output shows: "Final Decision → mRS = 1 → Favourable".

```
final_mrs = modelA_mrs if modelA_mrs == modelB_mrs else max(modelA_mrs, modelB_mrs)
print(f"\n✅ Final Decision → mRS = {final_mrs} → {'Favourable' if final_mrs <= 2 else 'Unfavourable'}")
```

The output of the cell shows: "Final Decision → mRS = 1 → Favourable".

REFERENCES:

- [1] K. Balasubramanian, S. Balakrishnan, and K. Sivalingam, “Automated stroke prediction using machine learning: An explainable and exploratory study with a web app,” *Journal of Medical Systems*, vol. 46, no. 5, p. 22, 2022.
doi: 10.1007/s10916-022-01813-1.
- [2] H. Barua, R. Bhowmick, M. N. S. Hassan, M. A. H. Akhand, and K. H. Mahmood, “AI-based stroke disease prediction system using ECG and PPG bio-signals,” *Bioengineering*, vol. 10, no. 9, pp. 1–18, 2023. doi: 10.3390/bioengineering10090994.
- [3] M. Chen, Y. Hao, Y. Cai, Y. Wang, and H. Su, “Innovations in stroke identification: A machine learning-based diagnostic model using neuroimages,” *IEEE Access*, vol. 10, pp. 123456–123468, 2022. doi: 10.1109/ACCESS.2022.3154215.
- [4] W. Ciezobka, J. Falcó-Roget, C. Koba, and A. Crimi, “End-to-end stroke imaging analysis using effective connectivity and interpretable artificial intelligence,” *IEEE Access*, vol. 13, pp. 10227–10241 2025. Doi: 10.1109/ACCESS.2025.3529179.
- [5] J. Galvez and L. B. Holder, “Evaluating performance of EEG data-driven machine learning for traumatic brain injury classification,” *Biomedical Signal Processing and Control*, vol. 58, p. 101837, 2020. Doi: 10.1016/j.bspc.2020.10183

- [6] A. Gupta, D. L. Lam, and M. D. Kuo, "Machine learning and acute stroke imaging," *AJR. American Journal of Roentgenology*, vol. 214, no. 1, pp. 144–151, 2020. doi: 10.2214/AJR.19.21321.
- [7] M. A. Hossen, N. A. M. Isa, and R. M. Noor, "A survey on genetic algorithm based feature selection for disease diagnosis system," *Informatics in Medicine Unlocked*, vol. 36, 2024, Art. no. 101236. doi: 10.1016/j.imu.2024.101236.
- [8] A. Javeed, A. L. Dallora, J. S. Berglund, A. Ali, P. Anderberg, and L. Ali, "Predicting dementia risk factors based on feature selection and neural networks," *Computers, Materials & Continua*, vol. 75, no. 2, pp. 2492–2508, 2023. doi: 10.32604/cmc.2023.033783.
- [9] S. Kumar and R. M. Thomas, "Machine learning algorithms versus thresholding to segment ischemic regions in patients with acute ischemic stroke," *Journal of Digital Imaging*, vol. 34, pp. 895–905, 2021. doi: 10.1007/s10278-021-00432-6.
- [10] S. Mehta, V. Sharma, and A. Arora, "Multi-task heterogeneous ensemble learning-based cross-subject EEG classification under stroke patient domain," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 31, pp. 45–55, 2023. doi: 10.1109/TNSRE.2023.3246824.
- [11] S. Rajalakshmi, A. Vadivel, and D. Arivazhagan, "An improved

concatenation of deep learning models for predicting and interpreting ischemic stroke,” *Journal of King Saud University - Computer and Information Sciences*, vol. 36, no. 2, pp. 127–137, 2024. doi: 10.1016/j.jksuci.2023.101107.

[12] T. P. Singh and D. J. Kim, “A machine learning approach to detect the brain stroke disease,” in *Proc. International Conference on Information Technology (ICIT)*, Bhubaneswar, India, 2022, pp. 1–6. doi: 10.1109/ICIT55440.2022.9995612.

[13] M. Sharma et al., “Decoding finger tapping with the affected hand in chronic stroke patients during motor imagery and execution,” *Frontiers in Neuroscience*, vol. 16, 2022. doi: 10.3389/fnins.2022.834816.

[14] R. Sharma, P. Verma, and A. Singh, “Analyzing the performance of stroke prediction using machine learning algorithms,” *International Journal of Advanced Computer Science and Applications*, vol. 13, pp. 525–534, 2022. doi: 10.14569/IJACSA.2022.0130762 .

[15] Y. D. Zhang et al., “An integrated machine learning approach to stroke prediction,” *Frontiers in Neuroscience*, vol. 16, 2022. doi: 10.3389/fnins.2022.843224.

NSCET/ICRTT - 2K25 / 42

Theni Melapettai Hindu Nadargal Uravinmurai

NADAR SARASWATHI COLLEGE OF ENGINEERING & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai : An ISO 9001 : 2015 Certified Institution
Vadapudupatti, Annanji (PO), Theni - 625531.

Accredited by NAAC with 'A' Grade

International Conference on Recent Trends in Technology

"ICRTT - 2k25"

Departments of AI&DS, CSE, ECE, EEE, and IT

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/Mr/Mrs/Ms. Santhiya K
from Chettinad College of Engineering and Technology has
presented a paper entitled Multi-Modal CNN Based Stroke Detection from CT Images and Clinical Data
in the International
Conference on Recent Trends in Technology "ICRTT - 2k25" held on 02nd May 2025 at Nadar Saraswathi College of
Engineering & Technology, Theni. We appreciate the valuable contribution and wish continued success in future
endeavours.

Dr.J.Mathalal Raj, M.E., Ph.D.,
Head Incharge - CSE, NSCET.

Dr.T.Venish Kumar, M.E., Ph.D.,
Head Incharge - ECE, NSCET.

Dr.R.Athilingam, M.E., Ph.D.,
Head Incharge - EEE, NSCET.

Dr.M.Sathya, M.Sc., M.B.A., Ph.D.,
Vice Principal & R&D - IT, NSCET.

Mr.L.S.Vigneh, M.E.,
Head Incharge - AI&DS, NSCET.

Dr.C. Mathalal Sundaram, M.E., M.B.A., Ph.D.,
Principal, NSCET.

Er.S.Naveen Ram, B.E., M.B.A.,
Joint Secretary, NSCET.

Mr.A.S.R.Maheswaran, B.Sc.,
Secretary, NSCET.

Mr.A.Rajkumar, B.A.,
Secretary, NSCET.

NSCET/ICRTT - 2K25 / 43

Theni Melapettai Hindu Nadargal Uravinmurai

NADAR SARASWATHI COLLEGE OF ENGINEERING & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai : An ISO 9001 : 2015 Certified Institution
Vadapudupatti, Annanji (PO), Theni - 625531.

Accredited by NAAC with 'A' Grade

International Conference on Recent Trends in Technology

"ICRTT - 2k25"

Departments of AI&DS, CSE, ECE, EEE, and IT

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/Mr/Mrs/Ms. Sarumathi A
from Chettinad College of Engineering and Technology has
presented a paper entitled Multi-Modal CNN Based Stroke Detection from CT Images and Clinical Data
in the International
Conference on Recent Trends in Technology "ICRTT - 2k25" held on 02nd May 2025 at Nadar Saraswathi College of
Engineering & Technology, Theni. We appreciate the valuable contribution and wish continued success in future
endeavours.

Dr.J.Mathalal Raj, M.E., Ph.D.,
Head Incharge - CSE, NSCET.

Dr.T.Venish Kumar, M.E., Ph.D.,
Head Incharge - ECE, NSCET.

Dr.R.Athilingam, M.E., Ph.D.,
Head Incharge - EEE, NSCET.

Dr.M.Sathya, M.Sc., M.B.A., Ph.D.,
Vice Principal & R&D - IT, NSCET.

Mr.L.S.Vigneh, M.E.,
Head Incharge - AI&DS, NSCET.

Dr.C. Mathalal Sundaram, M.E., M.B.A., Ph.D.,
Principal, NSCET.

Er.S.Naveen Ram, B.E., M.B.A.,
Joint Secretary, NSCET.

Mr.A.S.R.Maheswaran, B.Sc.,
Secretary, NSCET.

Mr.A.Rajkumar, B.A.,
Secretary, NSCET.

NSCET/ICRTT - 2K25 / 44

Theni Melapettai Hindu Nadargal Uravinmurai

NADAR SARASWATHI COLLEGE OF ENGINEERING & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai : An ISO 9001 : 2015 Certified Institution
Vadapudupatti, Annanji (PO), Theni - 625531.

Accredited by NAAC with 'A' Grade

International Conference on Recent Trends in Technology

"ICRTT - 2k25"

Departments of AI&DS, CSE, ECE, EEE, and IT

CERTIFICATE OF PARTICIPATION

This is to certify that Dr/Mr/Mrs/Ms. Kaniska M P
from Chettinad College of Engineering and Technology has
presented a paper entitled Multi-Modal CNN Based Stroke Detection from CT Images and Clinical Data
in the International
Conference on Recent Trends in Technology "ICRTT - 2k25" held on 02nd May 2025 at Nadar Saraswathi College of
Engineering & Technology, Theni. We appreciate the valuable contribution and wish continued success in future
endeavours.

Dr.J.Mathalal Raj, M.E., Ph.D.,
Head Incharge - CSE, NSCET.

Dr.T.Venish Kumar, M.E., Ph.D.,
Head Incharge - ECE, NSCET.

Dr.R.Athilingam, M.E., Ph.D.,
Head Incharge - EEE, NSCET.

Dr.M.Sathya, M.Sc., M.B.A., Ph.D.,
Vice Principal & R&D - IT, NSCET.

Mr.L.S.Vigneh, M.E.,
Head Incharge - AI&DS, NSCET.

Dr.C. Mathalal Sundaram, M.E., M.B.A., Ph.D.,
Principal, NSCET.

Er.S.Naveen Ram, B.E., M.B.A.,
Joint Secretary, NSCET.

Mr.A.S.R.Maheswaran, B.Sc.,
Secretary, NSCET.

Mr.A.Rajkumar, B.A.,
Secretary, NSCET.

